

Use of Interpolating Private Blockchains to Store and Analyze Genetic Data

Introduction

Currently there is a huge gap between the technologies involved in genetics and their application in human health. From privacy to public policy, concerns about the gathering and application of sequencing data and technologies make widespread application of these systems virtually unworkable. This is not to mention the various moral, sociological, practical and VAST technological challenges to overcome for full-scale IoT health monitoring to come into effect. Blockchain-based technologies have the potential to provide SHA 256 encryption and zero-knowledge proofs to fully anonymize data while profiting on the practical advantages of decentralized systems.

Advantages to a Blockchain-Based Approach

The first issue that must be addressed is: what exactly are the advantages of using a blockchain-based system as opposed to linked, centralized databases? The various obstacles described above are formidable, yet still possible to overcome. If blockchains do not provide some inherent benefit over traditional centralized systems of data, then it seems hardly worth the effort and investment for unnecessary "blockchainization".

Rationale of a blockchain-based approach:

1. The traditional negatives of centralized data: bottlenecking, downtime, and monopolization of information.
 - As anyone who has had the misfortune to use BLAST can attest, centralized data systems can constrict data analysis to the level of the data host's native capabilities. Blockchains remove this constriction by using cutting-edge scaling technologies and operating in a free-market environment. These two combine to provide a genuinely efficient data repository.
2. Virtually instantaneous consensus using the latest genomics research and data
 - Whereas a centralized entity (e.g. NCBI) requires vast effort and forced centralization to maintain a relatively modern data set, a blockchain can be edited/audited by anyone, anywhere, at any time. This ensures that the data stored is updated to the millisecond and is held to the highest level of accuracy.
3. Interaction between private and public data to produce a consumer-relevant result (i.e. usability)
 - If I'm being honest, all these reasons do not translate to showing how

- a smart contract is *absolutely* necessary. My system still requires an input of a sequenced genome, which you'd have to get done using conventional methods. And also, although I trash it a lot in this paper, BLAST does a perfectly good job at this. If you're really dedicated to using it, BLAST can really be an incredible tool for genomic medicine.
- But therein lies the problem—it's a *tool*. Tools are only as good as the users wielding them. BLAST and other such genomic analyzing services require a lot of training, attention, and even a base level of intelligence to use. It is folly to think that any more than a very tiny minority of people would ever find it worthwhile to use BLAST for their own health analysis at all, let alone at the scale discussed above. Tools are a necessary first stage in human advancement in a field, but we're rapidly approaching the time when we need the machines to handle the grunt work.
 - Today, going from a genome to a diagnosis requires many stages and professionals from a variety of different fields. An individual would have to work with a genomic scientist to start the request, who would interact with an engineer to build/maintain the machine, who would work with a software designer to sort the data, who would communicate with another genomic scientist who would validate the results, who would communicate these results to a medical practitioner who is familiar with genomic analyses, who would communicate the results to the family doctor, who would then—despite knowing little to nothing about the process by which the results were obtained— finally have to explain this information to our poor, beleaguered, even *less* informed individual.
 - In contrast, a genomic blockchain would automate every stage of this process, from the sequenced genome (or even earlier, if cheap sequencers become much more widespread) to a medical diagnosis. All of these steps would be unnecessary; a sequence would be translated into a diagnosis with a single transaction, which itself can be automated. The individual himself has to do *nothing*—and THAT is the key to this blockchain's success. As a wise person once said, it never pays to bet against human laziness.

Centralized services (e.g. 23andMe) haven't taken off as a diagnostic tool because (1) of privacy concerns and (2) the data generated is static and 'dumb', unable to interact with Web 3.0 (or even Web 2.0). The results are usually binary and diagnostic on a basic level. What we need is a system that will keep our data private while constantly monitoring our genome against the latest advances in genomics. If in 10 years they discover another SNP that is associated with an orphan disease, 23andMe won't be able to follow up with you. A system that

repeatedly and automatically transacts on a blockchain to monitor your health is the only feasible way to monitor your health to the highest degree of thoroughness. The computational load is enormous in summation but small on the individual level. The necessary computations can be done on the host's computer and make constant monitoring relatively trivial.

NOTE: There is already a solution that attempts to harness the power of the blockchain for storing genetic information. It's called "**Genecoin**". This is not a direct competitor to my system because its main purpose is to *secure* the genomic data, not to actually use it in smart contracts. While privacy is an amazing benefit of blockchains, simply storing genetic information barely improves on the current state of affairs. If an individual *still* has no easy way to link genomic data to their own health, storing this data is pointless. And unless there is a robust ecosystem of smart contracts securing this data from the time of acquisition to its use in diagnosis the whole privacy aspect is rendered completely useless. What is the point of using full military-grade security if you have to disable it to make any sort of diagnosis?

NOTE 2: Much of this paper is dedicated to finding and reporting single-nucleotide polymorphisms (SNP's). The human genome is *far* more complex than that. I felt it reasonable to ignore other forms of genetic variation because adding them would require only simple modifications to the base protocol. The SNP stage is a proof-of-concept; broadening it out to accept different types of mutation will not require much additional effort.

Feasibility

It would be folly to suggest that this task is easy to accomplish. While the idea may seem trivial to those acquainted with biology and blockchains, the actual application faces some structural difficulties reflective of the medium on which they are being built.

For all their benefits, blockchains are unwieldily beasts. The data stored on a blockchain has to be constantly broadcast to each node hosted around the planet, not to mention the actual resource requirement for storing all those bits and bytes in so many different computers. Because of these limitations this technology is limited by the amount of memory that can be contributed.

If the human genome is distilled down into its most efficient form, it takes about 1.5 GB of pure raw data to store it electronically¹. I really don't know how data is stored in the blockchain, but taking into allowance some additional space for sorting and transaction information, let us assume the amount of space/ Σ (bandwidth) it would take to store/upload an entire human genome onto a

blockchain would be 2.5 GB^2 ³.

This would only represent a single version of the human genome. There is a ridiculous amount of variation between genomes with currently unknown effects. There are several ways to get around this:

1. Upload a full genome for every human on the planet. Obviously infeasible. Even if we were to ignore the fact that it would cost tens if not *hundreds* of trillions of dollars, the side chain would have to store over 16 exabytes of data. Seems a tad inefficient.
2. Update each person's account with their own data of sequenced non-conserved positions (assuming 99.99% conservation between all humans). Theoretically possible, but the data requirement is still measured in petabytes.
3. Decentralize the computation required for analysis to the query computer. There was something in Byzantium that allowed some of the computation needed for zk-snarks to happen on the transaction sender's computer. If this can be done for the Genome blockchain, the data requirement on the blockchain is significantly reduced (down to the low-gigabyte level—very manageable). The blockchain would be used to store the genome (genesis block), update the genome (hard forks to change genesis block or state chain, needs development), store SNP's (subsequent blocks), and compare the reference genome to patient samples (subsequent transactions in blocks alongside SNP's)

1 Source

2 This estimate is a throwaway assuming utilization of a general-purpose chain, like Ethereum. I assume that a private chain—especially a purpose-built one—can be modified to minimize excess data. Transaction costs can be kept very low, although I don't foresee that being an issue anyway thanks to improvements to the Ethereum blockchain (Byzantium, Metropolis, etc)

3 The extra size of the blockchain added on is highly dependent on the structure of the actual chain. The genesis blocks can be mined either as a few HUGE blocks with massive amounts of data (possibly whole chromosomes), or an ENORMOUS amount of tiny blocks, scaling down to even the single-nucleotide level. Optimization between those two extremes should be determined by the relative efficiencies of the sorting algorithms with each setup.

Execution

Transactions:

What would the blocks on this blockchain look like? They would contain transactions with individual addresses (representing a single human) spending ETH to run a boolean operation on this side chain(s). If "true", there are no malignant SNP's and this result is thrown up as a positive validation and trigger a cascade of events in all associated blockchains to indicate that everything is fine. If "False", the system will (1) spit out the nucleotide and position of the mutation, (2) check against every viewable "State" of the blockchain (if using state) or scan the second dataset (genome vs "SNP database"), and (3) if there is a recognizable deleterious mutation, the blockchain will activate certain processes on the main chain/other interpolated chains to inform you/your doctor/your pharmacist/your healthcare provider of the minimum necessary information. This should be done automatically, but it can be done manually by those who want to know more about their genome.

NOTE: For this to work efficiently, the whole genome analysis would have to be split into massively parallel tasks. Raiden is a pre-requisite.

Searching:

The first consideration for searching the genome must be "how can we compare an input data set to a reference set without sacrificing anonymity"? Computation on encrypted data is still in its infancy (homomorphic encryption), so we'll have to break encryption *somewhere*. How can we do this while still maintaining a presumption of anonymity?:

1. First, the reference genome should be stored off-chain. There's no need to waste space on the blockchain when there are already really good references stored elsewhere (such as NCBI or GeneCoin)
2. Compare the hash of the genome with the input. If there aren't any differences, there's no need to waste time/energy in the rest of the work.
3. If there *are* deviations, then we'd go back to the reference genome. The 30,000 known coding sequences can be split into a variable number of small, anonymous, randomized chunks. The frequency and character of these random splits can be customized however you wish. An oracle can be used to find the chunk corresponding to the SNP in the data set.
4. This is where I'm not 100% sure. Can you use zk-snarks to verify alignment (or lack thereof) between on- and off-chain data? Because if so, then this can be done to select the small region of the gene with the mutation. The downside is that I don't think we'd be able to use IPFS, as I think it'd be necessary to erase all prior history of data splits to ensure anonymity, unless you can hide your timestamp?
5. Last step, finding the single-nucleotide error. If we've spent this much

time and energy ensuring randomness and privacy, I think it should be fine for the user's computer to download the specific random chunk and run the alignment in-house. The final error sequence (an alphanumeric string) can then be uploaded for further analysis or simply stored in-house for future reference.

Now that this problem is (potentially solved), we can focus on the actual act of searching.

In order to take advantage of this system, there has to be some incredible sorting algorithms. General sorting algorithms may be enough, but the very unique nature of this data lends itself to simplification/efficiency through very specialized algorithms (in order):

1. Checksum—used to monitor length. If the sequences have the same length, it's probably an SNP. If they have different lengths, may be an indel.
2. SNP search. This can be done with one of the more general algorithms. The data in the SNP database should be organized properly to maximize efficiency in searching. Sorting of the SNP dataset can be done with quick sort. If this data is separated into chromosomes (like chapters) and ordered by nucleotide number, simple searching algorithms would suffice (Hash? Binary search?).
3. 2-nucleotide errors. Similar to SNP search. Would be far more complex, but I believe this is still within the capabilities of the system. Efficiencies here would be ideal.
4. 3+ nucleotide errors. Each SNP has a frequency of $1: 10^{-6}$. 3+ SNP's in a row are essentially impossible in a genome sample. The chance of this appearing in a genome is so low, it can actually be used as a check against sequencing errors
 - Insertions/deletions: Can be monitored with checksum. If the sequence is longer/shorter than expected, the sequence has an indel. Each successful match resets the initial position to the next nucleotide. Negative position matching should be impossible (or highly disfavored). In case of an unsuccessful match, check for known SNP's. If not, run a statistics experiment. Based on the frequency of SNP's in the nearby nucleotides and the sequence as a whole, determine the significance of nucleotide mismatches. If the value drops below a maintenance level, start a simultaneous redo of the SNP search with the position on the reference genome shifted by 1. If the SNP search returns more statistically significant results, the indel is finished. If not, process forward another nucleotide and redo. Consider setting up an indel database to search against.

5. Transposition—Specific optional program. Can be used for scientific applications or as a very specific diagnostic tool

Reporting:

Once the transactions have been executed and the data sorted through, the information gleaned has to be reported in a way that can be reported to users. First, we should consider how our data appears at this point:

- We would first have a simple boolean statement of alignment. If no significant deviations have been reported from the sequence it would just return a "true" and terminate. There could be systems that carry this information further, just as a check or for epidemiological studies, but for most purposes this would be the end of the line. If "false", of course, the rest of the information will be crucial
 - Next would come the error report. We probably don't have to get too fancy with this. The position, error type, and error reporting can just be squashed into a single small string. Example, a SNP in position 183 on chromosome 3, A→T, could be reported as 3.1619,A-T. An insertion of 3 nucleotides in the same position can be read as 3.1619,+ATG.
- Once this data is reported, it can be analyzed two separate ways. It can either be sent straight as-is to a smart contract to compute, or it can itself be sent out as a new transaction to the subject's computer. The idea behind the second method would be that a simple transaction of this sort can be used to modify a copy of the base genome held on the subject's own computer. This genome (or just transaction history, for easier analysis) can then be retained for any type of analysis that the subject wishes, or just kept for fun. For most individuals, the first method would be sufficient. The data can just be sent as out-of-context packets to smart contracts to spit out the result.

I've spoken a lot about computers "spitting out" the results, but have yet to really go into *how* these results might be "spat". My vision is to make use of a whole ecosystem of smart contracts (or side chains) that can perform various analyses and report different kind of results. Some examples:

- Health monitor—this is probably the easiest one to understand. The reporting system simply takes information fed to it by the alignment algorithms and compares it to a reference set of deleterious mutations. This would require some decent searching algorithms, but these days writing a good one is trivial. Once the alignment is complete, notable issues are flagged and a specific response (encoded by the health

monitor) is sent to the patient, health professional, or whoever.

- Epidemiology calculator—A nice one for public policy. This chain could either take the full results and map non-canonical genomic sequences, or just call for specific mutations and test their prevalence. This could be *incredibly* useful for forming public policy; I don't think there's a comparable system with this level of medical depth and computational automation. You could plug this information into *another* algorithm and use it to calculate the medical needs of the population, paving the way for truly smart universal health insurance
- Law enforcement—A dangerous road to walk down, but if done properly, system like this can bring to an end the days of local criminal databases and allow law enforcement officials to track down individual criminals by their specific genetic profile. With zk-snarks this can't be used to identify the criminal, but if their genetic profile is linked to other things (bank account, social security, etc), then this can be a very efficient way to track down a criminal automatically without any human input. But as I said, it's a dangerous path. Many safeguards will need to be used to prevent this from being abused by an oppressive and overzealous government.

Blockchain Structure

First off, even storing 1.5 GB of genomic DNA can make the blockchain too chunky. There are ways to access data stored off chain, so let's leave that off for now. Instead, the blockchain shall consist of the relevant SNP's.

Layer 1: The genome itself will be stored off-chain, either in a private server or some other off-chain solution (IPFS?), while the hash will be stored on the blockchain. If the sequence is in total alignment, it will return a successful value. If there is a misalignment, the hash will indicate as much and begin a second transaction.

Layer 2: The second layer is a data set stored either on the blockchain or on a private server or some other off-chain solution (IPFS?). If the data is stored off chain, this data can be oracalized with the inout data. If on-chain, it's still a read-only transaction. This layer

Layer 3: The third layer is the set of transactions that occur on the blockchain. This should be kept to a minimum, ideally no more than the two stated queries, two boolean returns, and a string return for SNP's. Modifications to the base genome or the SNP library should be done by either directly modifying the private database or by forking the blockchain (depending on how they're stored).

Additional notes:

- Blocksize should be large for the genesis block. Keeping all of one data type would ease searching and make everything so much simpler. Future blocks should be small, but not extremely so. The block size and time should vary to ensure the highest level of throughput while still ensuring some delay to prevent orphaning. Perhaps a future version of the internet will be able to achieve consensus on the millisecond timescale, but it makes sense to use a block time in the second range (15-ish seconds?).
- In order for this to scale to support large-scale computational analysis, full-genome (or exome) sequence alignment, sharding will have to be implemented. It would make no sense to build a system like this unless the alignment for each gene length can be done in a massively parallel manner. This strengthens the hard limit for the block time listed in the previous point, as it needs to be long enough to allow for consensus to be achieved.
- State chain for the SNP's is something to be considered. If we can store multiple "States" for each nucleotide with an SNP, it might potentially reduce the difficulty of searching and storing data. First the sequence will have to be identified as a SNP only, not some other kind of variation (those will be dealt with separately). If an SNP, alignment will commence with the 0 state, and each failed alignment will cause the state of the system to cycle up one (i.e. state 0, state 1, state 2, state 3). State chains explained a bit on [Github](#)
- I'm still considering purging. Transaction history is not necessary, especially if this is meant for every human to constantly monitor their genetic health with the latest research.

Infrastructure

How is this going to go from some nerd's computer to a real system?

We need a bunch of university people to humor us by donating computer space, 23andMe, etc. would jump at the change to multiply their business, so they can easily handle the sequencing work (this is probably going to be an under-the-hood system; anyone *can* use it, but likely health professionals will be teaming up with sequencing monkeys with the blockchain acting as an automatic intermediary)

NCBI will be more than happy to give free computing power if it means they don't have to deal with the operational headache of BLAST, etc.

